마이칩 과제

이현서

1.Talks 디렉토리에서 인공지능과 대화한 내용을 검토

Q14와 Q11 디렉토리의 인공지능 대화 내용을 확인해보니, 시스템C 테스트벤치와 SDL2 그래픽 환경을 활용하여 코드를 검증하고 실제 작동 가능한 칩 제작을 준비하는 과정이 담겨 있었다. RTL 코드가 합성되면 넷리스트를 거쳐 ETRI 0.5um 표준셀 레이아웃으로 변환되는데, 이때 할당된 칩 면적과 배치 밀도를 초과하면 제작 후 동작하지 않는 문제가 발생할 수 있어 사전 검증이 매우 중요함을 알 수 있었다.

특히 학습 과정에서 면적을 제어하는 부분이 핵심적으로 다루어졌다. Q11에서는 스프라이트와 폰트 데이터가 ROM 대신 조합논리회로로 자동 합성되면서 면적 부담이 커지는 점을 확인했고, Q14에서도 5×5 중첩 반복문이 회로를 평면에 펼쳐(Unrolling) 면적을 급격히 키우는 요인으로 분석되었다. 여기에 연산 과정의 곱셈기 역시 상당한 면적을 차지하는 것으로 나타났다.

이에 따라 Q14에서는 무거운 곱셈 연산을 비트 시프트 연산과 카운터 구조로 대체하여 회로 규모를 대폭 줄였으며, 실제 소스 코드를 수정함으로써 면적을 80% 이상 절감할 수 있음을 확인했다. 이처럼 반도체 설계에서는 기능 구현뿐만 아니라 실제 공정 규격에 맞추어 면적과 자원을 최적화하는 과정이 필수적이라는 점을 배울 수 있었다.

2.simulation 디렉토리 에서 make를 실행하면 몇가지 상이한 베릴로그 RTL비교

simulation 디렉토리에서 Makefile을 분석한 결과, # Build Rules의 all 조건문을 통해 0부터 4까지 총 5개의 베릴로그 소스코드 중 하나를 심볼릭 링크(ln -s) 방식으로 선택하여 상위 디렉토리의 최상위 모듈인 space_invaders_engine_glcd.v 파일과 연결되어있었다.

Q13 환경에서 0번부터 4번까지의 RTL 코드를 각각 실행하여 시뮬레이션을 진행한 결과, 버전별로 뚜렷한 동작 차이와 오류가 나타났다. 0번 파일은 우주선이 벽을 뚫고 순간이동하는 현상이 발생했고, 1번 파일은 총알이 화면에 아예 보이지 않았으며, 2번과 3번 파일은 스페이스바를 한 번만 눌렀음에도 총알이 연달아 다중 발사되어 적들이 한 줄씩 통째로 소멸하는 문제가 발생했다. 또한 0번부터 3번까지는 공통적으로 적의 움직임과 공격 속도가 매우 느렸고, 파형 분석 결과 DPI-C WAVE와 SYSTEMC 파형 모두에서 총알 관련 파형인 inv_bullet이 측정되지 않았다. 반면 4번 파일에 이르러서야 게임이 정상적으로 구동되었으나, 창 크기를 조절해도 게임 화면 크기가 커지지 않는 공통적인 현상이 확인되었다.

코드 구조를 비교해 보면 버전이 바뀜에 따라 인터페이스 버스 선언부의 데이터 폭이 변경되고, 상태를 저장하는 current_state와 next_state 레지스터 변수가 2비트에서 5비트로 확장되었으며, 그룹 위치 관리를 위한 group_x와 group_y 변수가 추가되는 등 상태 관리 로직에 변화가 있었다.

이어진 Q14 실습에서는 실행이 원활하지 않던 코드를 직접 수정하였다. 2번과 3번 파일은 초기 실행 시 컴파일 에러가 발생했는데, 이는 c, t_invader_x, t_invader_y 등의 필수 변수가 선언되지 않았거나 for 반복문 내 비트 지정 상수 연산으로 인해 -1 언더플로우가 발생했기 때문이었다. 이를 해결하기 위해 루프 변수를 integer 타입으로 변경하고 반복 범위의 숫자를 4, 0, 1 등으로 수정하였으며, 곱셈기 연산을 시프트 연산자로 변경하고 중첩 반복문 구조를 개선하여 정상 작동을 유도했다. 4번 파일은 수정 없이 곧바로 작동하였고, 0번 파일을 제외한 모든 버전에서는 스코어 표시 기능이 누락되어 있음을 확인하였다.